**PRN: 260840125033 260840125041**

**Assignment 3**

**Use appropriate collection to store Songs**
**Song has a serialno, title, set of artists ,release year and rating**

**On the collection of Songs perform follwoing operations**

**1. Add/Remove/Update and Display list of the songs**
**2. Sort songs by title**
**3. Search song by title**
**4. Create a list of songs released in the current year**
**5. Create a list of songs by specific artist**
**6. Create a listing to show song title and release year**
**7. Display the song with lowest and highest rating**

---
**// song class**

```
package javaassignment3;
import java.util.Set;

class Song {

    private int serialNo;
    private String title;
    private Set<String> artists;
    private int releaseYear;
    private double rating;

    public Song(int serialNo, String title, Set<String> artists,
            int releaseYear, double rating) {

        this.serialNo = serialNo;
        this.title = title;
        this.artists = artists;
        this.releaseYear = releaseYear;
        this.rating = rating;
    }

    // Getters
    public int getSerialNo() {
        return serialNo;
    }

    public String getTitle() {
        return title;
    }

    public Set<String> getArtists() {
        return artists;
    }
```

```
    public int getReleaseYear() {
        return releaseYear;
    }

    public double getRating() {
        return rating;
    }

    // Setters
    public void setSerialNo(int serialNo) {
        this.serialNo = serialNo;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public void setArtists(Set<String> artists) {
        this.artists = artists;
    }

    public void setReleaseYear(int releaseYear) {
        this.releaseYear = releaseYear;
    }

    public void setRating(double rating) {
        this.rating = rating;
    }

    @Override
    public String toString() {
        return "Song{" +
                "serialNo=" + serialNo +
                ", title='" + title + '\'' +
                ", artists=" + artists +
                ", releaseYear=" + releaseYear +
                ", rating=" + rating +
                '}';
    }
}
```

**// MediaApp.java**

```
package javaassignment3;

import java.util.*;

public class MediaApp {

        static List<Song> songs = new ArrayList<>();
```

```
	// 1.1 ADD SONG
	public static void addSong(Song song) {
		songs.add(song);				// using list method
	}

	// 1.2 REMOVE SONG
	public static void removeSong(int serialNo) {
		Iterator<Song> iterator = songs.iterator(); //
		while (iterator.hasNext()) {
			Song song = iterator.next();

			if (song.getSerialNo() == serialNo) {
				iterator.remove();
				System.out.println("Song removed.");
				return;
			}
		}

		System.out.println("Song not found.");
	}

	//1.3 UPDATE SONG
	public static void updateSong(int serialNo, String title, Set<String> artists, int releaseYear,
double rating) {
		for (Song song: songs) {
			if (song.getSerialNo() == serialNo) {
				song.setTitle(title);
				song.setArtists(artists);
				song.setReleaseYear(releaseYear);
				song.setRating(rating);
				System.out.println("Song updated.");
				return;
			}
		}

		System.out.println("Song not found.");
	}

	//1.4 DISPLAY SONGS
	public static void displaySongs() {
		for(Song song: songs) {
			System.out.println(song);
		}
	}

	//2 SORT SONGS BY TITLE
	public static void sortByTitle() {
		songs.sort(
	   Comparator.comparing(Song::getTitle)
	);

	System.out.println("Songs sorted by title.");
```

```
        }

        //3. SEARCH SONG BY TITLE
        public static void searchByTitle(String title) {

    for (Song song : songs) {

        if (song.getTitle().equalsIgnoreCase(title)) {

            System.out.println("Song found:");
            System.out.println(song);

            return;
        }
    }

    System.out.println("Song not found.");
}

        // 4. SONGS RELEASED IN CURRENT YEAR
  public static List<Song> songsOfCurrentYear() {

    int currentYear = Year.now().getValue();
    List<Song> result = new ArrayList<>();

    for (Song song : songs) {

        if (song.getReleaseYear() == currentYear) {
            result.add(song);
        }
    }

    return result;
}

// 5. SONGS BY SPECIFIC ARTIST
  public static List<Song> songsByArtist(String artist) {

    List<Song> result = new ArrayList<>();

    for (Song song : songs) {

        if (song.getArtists().contains(artist)) {
            result.add(song);
        }
    }

    return result;
}


  // 6. TITLE + RELEASE YEAR
```

```
    public static void titleAndYearListing() {

        for (Song song : songs) {
            System.out.println(song.getTitle() + " - " + song.getReleaseYear());
        }
    }


    // 7. LOWEST AND HIGHEST RATING
    public static void lowestHighestRating() {

        if (songs.isEmpty()) {
            System.out.println("No songs available.");
            return;
        }

        Song lowest = songs.get(0);
        Song highest = songs.get(0);

        for (Song song : songs) {

            if (song.getRating() < lowest.getRating()) {
                lowest = song;
            }

            if (song.getRating() > highest.getRating()) {
                highest = song;
            }
        }

        System.out.println("Lowest rated song:");
        System.out.println(lowest);

        System.out.println("\nHighest rated song:");
        System.out.println(highest);
    }

// MAIN
    public static void main(String[] args) {

        // Artists
        Set<String> artists1 = new HashSet<>();
        artists1.add("Imagine Dragons");

        Set<String> artists2 = new HashSet<>();
        artists2.add("Ed Sheeran");

        Set<String> artists3 = new HashSet<>();
        artists3.add("Coldplay");
        artists3.add("The Chainsmokers");

        Set<String> artists4 = new HashSet<>();
```

```
artists4.add("Adele");


// Songs
Song s1 = new Song(1, "Believer", artists1, 2017, 4.8 );

Song s2 = new Song(2, "Perfect", artists2, 2017, 4.7);

Song s3 = new Song( 3, "Something Just Like This", artists3, 2017, 4.6 );

Song s4 = new Song( 4, "Hello", artists4, 2015, 4.5 );


// Add songs
addSong(s1);
addSong(s2);
addSong(s3);
addSong(s4);


// 1. Display
System.out.println("===== ALL SONGS =====");
displaySongs();


// 2. Sort
System.out.println("\n===== SORTED BY TITLE =====");
sortByTitle();
displaySongs();


// 3. Search
System.out.println("\n===== SEARCH =====");
searchByTitle("Believer");


// 4. Current year
System.out.println("\n===== CURRENT YEAR SONGS =====");

List<Song> currentYearSongs = songsOfCurrentYear();  // songsOfCurrentYear() returns a list

for (Song song : currentYearSongs) {
   System.out.println(song);
}


// 5. Songs by artist
System.out.println("\n===== SONGS BY ARTIST =====");

List<Song> coldplaySongs =
       songsByArtist("Coldplay");                  //  songsByArtist() returns a list
```

```
        for (Song song : coldplaySongs) {
            System.out.println(song);
        }


        // 6. Title + year
        System.out.println("\n===== TITLE + YEAR =====");

        titleAndYearListing();


        // 7. Lowest + highest
        System.out.println("\n===== RATINGS =====");

        lowestHighestRating();


        // Remove
        System.out.println("\n===== REMOVE =====");

        removeSong(4);

        displaySongs();


        // Update
        System.out.println("\n===== UPDATE =====");

        Set<String> updatedArtists = new HashSet<>();
        updatedArtists.add("Adele");

        updateSong(
                3,
                "Something Just Like This - Updated",
                updatedArtists,
                2017,
                4.9
        );

        displaySongs();
    }
}
```

**Output**

```
===== ALL SONGS =====
Song{serialNo=1, title='Believer', artists=[Imagine Dragons], releaseYear=2017, rating=4.8}
Song{serialNo=2, title='Perfect', artists=[Ed Sheeran], releaseYear=2017, rating=4.7}
Song{serialNo=3, title='Something Just Like This', artists=[The Chainsmokers, Coldplay],
releaseYear=2017, rating=4.6}
```

Song{serialNo=4, title='Hello', artists=[Adele], releaseYear=2015, rating=4.5}

===== SORTED BY TITLE =====
Songs sorted by title.
Song{serialNo=1, title='Believer', artists=[Imagine Dragons], releaseYear=2017, rating=4.8}
Song{serialNo=4, title='Hello', artists=[Adele], releaseYear=2015, rating=4.5}
Song{serialNo=2, title='Perfect', artists=[Ed Sheeran], releaseYear=2017, rating=4.7}
Song{serialNo=3, title='Something Just Like This', artists=[The Chainsmokers, Coldplay], releaseYear=2017, rating=4.6}

===== SEARCH =====
Song found:
Song{serialNo=1, title='Believer', artists=[Imagine Dragons], releaseYear=2017, rating=4.8}

===== CURRENT YEAR SONGS =====

===== SONGS BY ARTIST =====
Song{serialNo=3, title='Something Just Like This', artists=[The Chainsmokers, Coldplay], releaseYear=2017, rating=4.6}

===== TITLE + YEAR =====
Believer - 2017
Hello - 2015
Perfect - 2017
Something Just Like This - 2017

===== RATINGS =====
Lowest rated song:
Song{serialNo=4, title='Hello', artists=[Adele], releaseYear=2015, rating=4.5}

Highest rated song:
Song{serialNo=1, title='Believer', artists=[Imagine Dragons], releaseYear=2017, rating=4.8}

===== REMOVE =====
Song removed.
Song{serialNo=1, title='Believer', artists=[Imagine Dragons], releaseYear=2017, rating=4.8}
Song{serialNo=2, title='Perfect', artists=[Ed Sheeran], releaseYear=2017, rating=4.7}
Song{serialNo=3, title='Something Just Like This', artists=[The Chainsmokers, Coldplay], releaseYear=2017, rating=4.6}

===== UPDATE =====
Song updated.
Song{serialNo=1, title='Believer', artists=[Imagine Dragons], releaseYear=2017, rating=4.8}
Song{serialNo=2, title='Perfect', artists=[Ed Sheeran], releaseYear=2017, rating=4.7}
Song{serialNo=3, title='Something Just Like This - Updated', artists=[Adele], releaseYear=2017, rating=4.9}

===== GROUP SONGS BY YEAR =====
Year: 2017
  -> Song{serialNo=1, title='Believer', artists=[Imagine Dragons], releaseYear=2017, rating=4.8}
  -> Song{serialNo=2, title='Perfect', artists=[Ed Sheeran], releaseYear=2017, rating=4.7}

-> Song{serialNo=3, title='Something Just Like This - Updated', artists=[Adele], releaseYear=2017, rating=4.9}

---

## Concepts Learned:

**Iterator:**
Its a design pattern used to step through a collection of items (like a list or a set) one by one without exposing how collections is internally organized and stored for accessing.

- songs.iterator() creates a new iterator object positioned just before the first element of the songs collection.
- songs.hasNext() checks if there is any element next to current cursor. Return true or false respectively.
- songs.next() returns the next element and moves the cursor to next position
- songs.remove() is a safe modifier that removes the last element returned by next(). It keeps iterator's internal counter synced with collection size, otherwise a normal for loop will return ConcurrentModificationException

**Comparator:**
A Comparator is an interface used to define a custom sorting rule for objects. Otherwise java doesn't how which object's attribute it should sort it on.
Using it we can apply different rules to compare, e.g. based on serialNo, alphabetical or reverse alphabetical order, or even mulitple attributes.

Comparator.comparing(Song::getTitle) first extract Title for every Song object and applied the comparision rule, then it passes that rule directly into songs.sort() which uses TimSort Algorithm to sort collection of objects.